



Universidad de Jaén

## **Bloque 4**

### **Diseño de Software**



**EPS**  
Escuela Politécnica  
Superior de Jaén

## Tema 9. Fundamentos del Diseño de Software

L. Alfonso Ureña López    Eugenio Martínez Cámara

Departamento de Informática  
Universidad de Jaén

Fundamentos de Ingeniería del Software  
Grado en Ingeniería Informática  
2º Curso

# Contenidos

## 1 Objetivos

## 2 Introducción

- Definición de diseño
- Dónde se encuadra
- Cómo se desarrolla

## 3 De los requisitos al diseño

- Análisis vs Diseño
- Analista vs Diseñador
- Dónde termina el análisis y dónde comienza el diseño

## 4 Diseño tradicional vs diseño iterativo

- Diseño tradicional
- Diseño iterativo

## 5 Conceptos de diseño

## 6 Modelo del diseño

- Características
- Actividades de diseño

# Objetivos

- Conocer el concepto de diseño y dónde situarlo en función del ciclo de vida del modelo de proceso a aplicar
- Conocer los distintos tipos de diseño en función del punto de vista de desarrollo
- Saber distinguir qué responsabilidades implican al análisis y cuáles al diseño
- Conocer cuándo termina la fase de análisis y cuándo comienza la de diseño
- Conocer los conceptos fundamentales relacionados con el diseño, tales como *abstracción, patrones, modularidad, independencia funcional, ocultamiento de información, refinamiento por pasos y clases del diseño*
- Conocer el conjunto de actividades a desarrollar para obtener un diseño completo y consistente

# Diseño



## Definición (Taylor, 1959)

*... el proceso de aplicar distintas técnicas y principios con el propósito de definir un dispositivo, proceso o sistema con suficientes detalles como para permitir su realización física*

# Dónde se encuadra

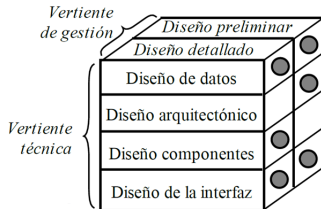
- Es la **primera fase** de construcción del sistema desde el punto de vista del **dominio de la solución** (sistema software):
  - El **Análisis** estudia el sistema a desarrollar desde el punto de vista del **dominio del problema**
- La **fase** concreta en que se desarrolla el diseño **depende** del **ciclo de vida** que se utilice

# Cómo se desarrolla



I El proceso de desarrollo depende del punto de vista

- Punto de vista de **gestión**:
  - Diseño preliminar
  - Diseño detallado
- Punto de vista **técnico**:
  - Diseño de datos
  - Diseño arquitectónico
  - Diseño a nivel de componentes
  - Diseño de la interfaz



# Análisis vs Diseño



El **diseño** plantea *cómo construir el sistema sin realmente construirlo* (Rumbaugh, 1997)

- El **análisis** identifica **qué** debe hacer el sistema
- El **diseño** especifica **cómo** debe hacerlo



# Analista vs Diseñador



## Analista

El **analista** busca **entender** la organización, sus requisitos y sus objetivos



## Diseñador

El **diseñador** busca **especificar un sistema** que se ajuste a la organización, provea sus requisitos eficazmente y permita alcanzar sus objetivos

## Ejemplo: la clase **Campaña**

- El **análisis** identifica el hecho de que la **clase Campaña** tiene el **atributo título**
- El **diseño** determina cómo la **clase** será **introducida** en el **sistema**, cómo se **visualizará** en pantalla y cómo se **almacenará** en la base de datos, además de identificar todos los **demás atributos** necesarios

# Dónde termina el análisis y dónde comienza el diseño

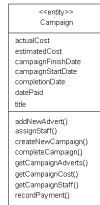
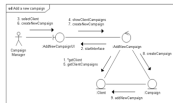
- En el **ciclo de vida en cascada** existe una **transición clara** entre ambas actividades
- En un **ciclo de vida iterativo**, el **análisis** de una parte (caso de uso) del sistema **precederá** a su **diseño**, pero el análisis y el diseño **podrían desarrollarse en paralelo**
- Es importante **distinguir ambas actividades** y la **mentalidad asociada** a cada una
- Necesitamos conocer **qué** hacer **antes** de decidir **cómo** hacerlo

## Requirements

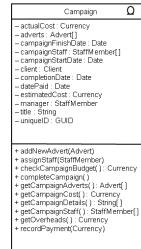


2. To record the details of each campaign for each client. This will include the title of the campaign, planned start and finish dates, estimated costs, budgets, actual costs and dates, and the current state of completion.

## Analysis

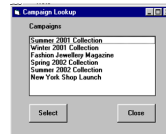


## Design



```

    CREATE TABLE Campaigns
    (VARCHAR(50) uniqueID PRIMARY KEY NOT NULL,
    FLOAT actualCost,
    DATE campaignFinishDate,
    DATE campaignStartDate,
    VARCHAR(50) clientID NOT NULL,
    DATE completionDate,
    DATE datePaid,
    FLOAT estimatedCost,
    VARCHAR(50) managerID,
    VARCHAR(50) title);
    CREATE INDEX campaign_idx ON Campaigns (clientID, managerID, title);
  
```



# Diseño tradicional

Desde el punto de vista tradicional, el desarrollo orientado a objetos (O. O.) se divide en las siguientes **actividades**:

- **Análisis orientado a objetos**

- Desarrollo de un **modelo O. O. del dominio de aplicación**
- Los **objetos** identificados reflejan las **entidades y operaciones** que se asocian con el problema a resolver

- **Diseño orientado a objetos**

- Desarrollo de un **modelo O. O. para diseñar los requisitos identificados**
- Los **objetos** están relacionados con la **solución del problema por resolver**

- **Programación orientada a objetos**

- Elección y uso de un **lenguaje de programación O. O.**

# Diseño tradicional



## Inconvenientes

- **Dirección del proyecto:** ¿existe un equilibrio correcto entre actividades?
- **Interacción del equipo:** análisis y diseño podría ser desarrollado por equipos independientes
- **Decisiones del cliente:** el cliente podría esperar una especificación del qué antes de aprobar el gasto en el diseño
- **Elección del entorno de desarrollo:** podría retrasarse hasta que finalice el análisis

# Diseño en el ciclo de vida iterativo

Actualmente se distinguen otro tipo de fases:

- **Planificación y Especificación de Requisitos**
  - Planificación, definición de requisitos, construcción de prototipos, etc.
- **Construcción:** se divide en:
  - **Análisis(o diseño de Alto Nivel):** se centra en la visión que tienen los actores desde el exterior (los casos de uso)
  - **Diseño(o diseño de Bajo Nivel):** el sistema definido en la fase anterior se especifica en detalle
  - **Implementación** de la especificación del diseño
  - **Pruebas:** comprobar que el software funciona correctamente y que satisface los requisitos del sistema
- **Instalación:** Puesta en marcha del sistema en su entorno previsto

# Diseño en el ciclo de vida iterativo



## Ventajas

- **Mitigación de riesgos:** facilidad para identificar riesgos en fases tempranas y tomar decisiones al respecto
- **Control de cambios:** sensible a cambios en los requisitos y su control adecuado
- **Aprendizaje del equipo:** todo el equipo puede estar involucrado desde el comienzo del proyecto
- **Calidad mejorada:** las pruebas se hacen desde el principio evitando su ejecución precipitada al final del proyecto



# Conceptos de diseño

- Repasemos algunos **conceptos fundamentales**, aplicables en la mayoría de las metodologías de desarrollo:
  - Abstracción
  - Uso de Patrones
  - Modularidad
  - Independencia funcional
  - Ocultamiento de información
  - Refinamiento por pasos
  - Clases de diseño
- Muchos de estos conceptos son **complementarios** y están muy **relacionados** unos con otros

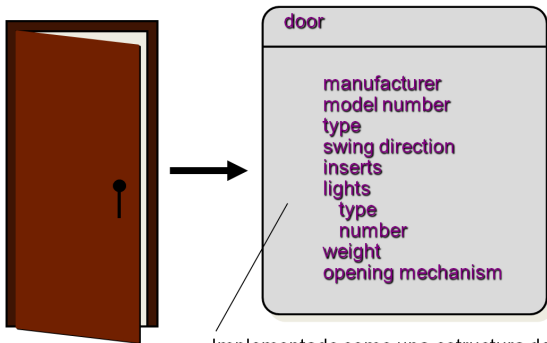
# Abstracción

- Una solución modular para cualquier problema plantea muchos **niveles de abstracción**:
  - En el **más elevado** se enuncia una **solución a grandes rasgos** para el problema
  - En **niveles más bajos** de abstracción se da una **descripción más detallada de la solución**
  - En el **nivel más bajo** de abstracción se plantea la **solución**, de modo que pueda **implementarse directamente**

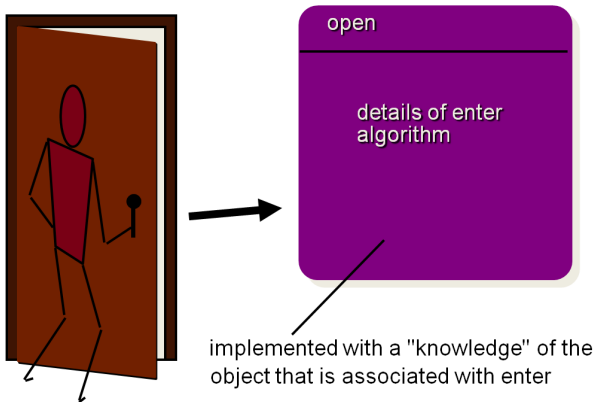
# Abstracción

- **Mecanismos de abstracción** en el diseño de software:
  - **Abstracción funcional:** secuencia de instrucciones con una función limitada y específica
  - **Abstracción de datos:** tipos de datos a utilizar, junto a sus operaciones permitidas. Encapsulado de datos

# Abstracción de datos



# Abstracción funcional



# Uso de patrones

- Un **patrón** es una **solución a un problema general**, utilizable para resolver un problema particular **ajustando sus atributos**
- **Tipos de patrones** según el **grado de abstracción**:
  - **Patrones arquitectónicos**: define la estructura general del software y las relaciones entre subsistemas y componentes del software
  - **Patrones de diseño**: consiste en un diagrama de objetos que ofrece una solución a un problema conocido y frecuente. Dicho diagrama está formado por un conjunto de objetos descritos por clases y las relaciones entre ellas.
  - **Patrones de código**: suelen implementar un elemento algorítmico o un componente, un protocolo de interfaz específico o un mecanismo de comunicación entre los componentes

# Patrones de diseño I

- Los **patrones** responden a problemas de diseño de aplicaciones en el marco de la programación O. O. Se trata de **soluciones conocidas y probadas** cuyo diseño proviene de la **experiencia** de los **programadores**
- Pueden describirse mediante una **plantilla**:
  - **Nombre del patrón**: preferentemente corto, pero descriptivo
  - **Intención**: describe el patrón y lo que hace
  - **Motivación**: proporciona un ejemplo del problema
  - **Estructura**: describe las clases que se requieren para implementar el patrón
  - **Participantes**: describe responsabilidades de clases necesarias para implementarlo

## Patrones de diseño II

- **Colaboraciones**: describe la colaboración entre participantes para llevar a cabo sus responsabilidades
- **Dominios de uso**: anota situaciones específicas de diseño a las que es aplicable
- **Patrones relacionados**: referencias cruzadas a otros patrones de diseño
- La **GoF** ofrece un catálogo de **23 patrones de diseño** organizados según su vocación:
  - Los patrones de **construcción** tienen como objetivo organizar la creación de objetos. Son **cinco**: Abstract Factory, Builder, **Factory Method**, Prototype y Singleton



## Patrones de diseño III

- Los patrones de **estructuración** facilitan la organización de la jerarquía de clases y de sus relaciones. Son **siete**: **Adapter**, Bridge, Composite, Decorator, Facade, Flyweight y Proxy
- Los patrones de **comportamiento** proporcionan soluciones para organizar las interacciones y para repartir el procesamiento entre los objetos. Son **once**: Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, **Template Method** y Visitor
- Veamos un **ejemplo** de cada tipo...

# El patrón Factory Method



## Descripción

El objetivo del patrón **Factory Method** es proveer un método abstracto de creación de un objeto delegando en las subclases concretas su creación efectiva



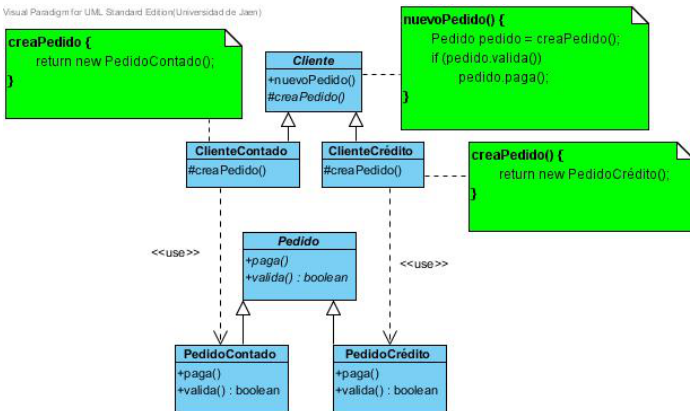
## Ejemplo: clientes y pedidos

La clase **Cliente** implementa el método **creaPedido** que debe crear el pedido. Existe la posibilidad de pagar los pedidos al contado o a crédito. En función de la naturaleza del cliente el método **creaPedido** debe crear una instancia de la clase **PedidoContado** o de la clase **PedidoCrédito**

# El patrón Factory Method

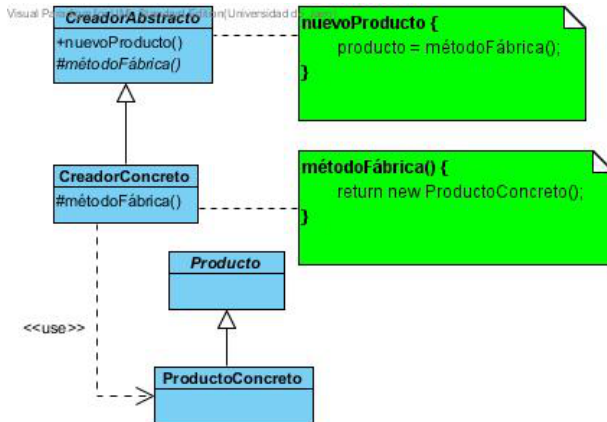
Diagrama de clases aplicado a los clientes y sus pedidos

Visual Paradigm for UML Standard Edition (Universidad de Jaén)



# El patrón Factory Method

## Diagrama de clases estructural



# El patrón Adapter



## Descripción

El objetivo del patrón **Adapter** es convertir la interfaz de una clase existente en la interfaz esperada por los clientes también existentes de modo que puedan trabajar de manera conjunta



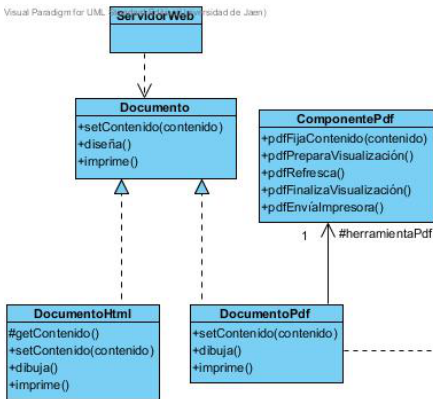
## Ejemplo: gestión de documentos

El **servidor Web** de un sistema de ventas crea y administra los documentos destinados a los clientes. Se define la interfaz **Documento** para realizar esta gestión. Se crea la clase **DocumentoHtml**, mientras que para la gestión de documentos pdf se dispone del componente **ComponentePdf**, cuya interfaz es muy diferente a la de la clase anterior

# El patrón Adapter

Diagrama de clases aplicado a los clientes y sus pedidos

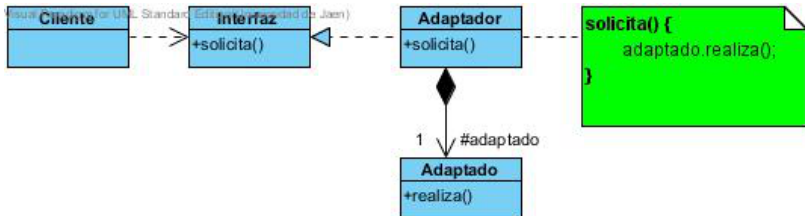
Visual Paradigm for UML Universidad de Jaén



```
setContenido(contenido) {
    herramientaPdf.pdfFijaContenido(contenido);
}
dibuja() {
    herramientaPdf.pdfPreparaVisualización();
    herramientaPdf.pdfRefresca();
    herramientaPdf.pdfFinalizaVisualización();
}
imprime {
    herramientaPdf.pdfEnvíaImpresora();
}
```

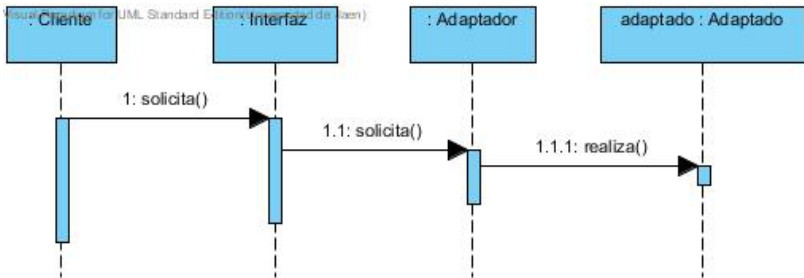
# El patrón Adapter

## Diagrama de clases estructural



# El patrón Adapter

## Diagrama de secuencia asociado





# El patrón Template Method



## Descripción

El patrón **Template Method** permite delegar en las subclases ciertas etapas de una de las operaciones de un objeto, quedando estas etapas descritas en las subclases

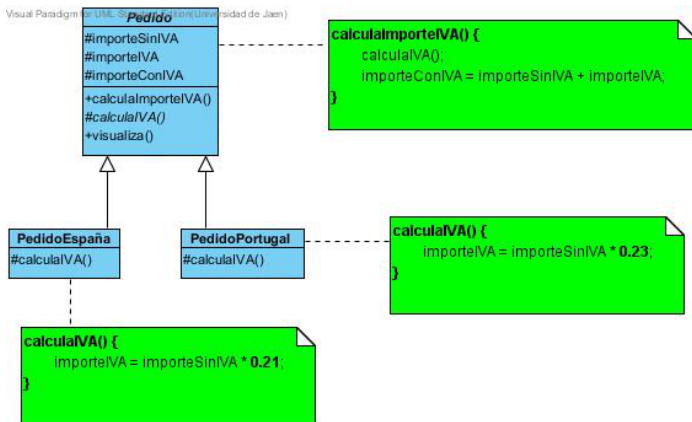


## Ejemplo: cálculo del IVA

Un sistema de **ventas** gestiona pedidos procedentes de España y Portugal. La diferencia entre ambas peticiones viene dada fundamentalmente por el tipo general del **IVA**, el cual actualmente está fijado en el **21 %** para **España** y el **23 %** para **Portugal**

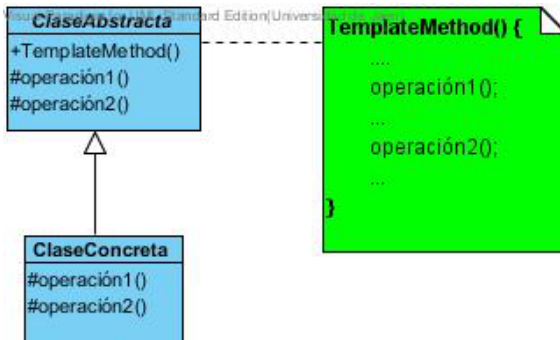
# El patrón Template Method

## Diagrama de clases aplicado al cálculo del IVA



# El patrón Template Method

## Diagrama de clases estructural



# Modularidad

- La **modularidad** es el *único atributo de un software que permite que un programa sea manejable intelectualmente* (Myers 1978)
- El software monolítico (sin modularidad) es difícil de seguir
- En un **diseño modular**, el software se divide en componentes con nombres independientes a los que se denomina **módulos**
- El **conjunto de módulos** satisfacen los **requisitos del problema**

# Propiedades

## Modularidad



### Propiedades de un sistema modular

- La **abstracción de un proceso** será un subsistema definido con **claridad**
- Las **funciones** tendrán un **propósito específico**
- Las funciones **sólo compartirán datos globales** y de forma selectiva
- **Encapsulación** de las instancias de los **tipos de datos** con sus **estructuras**

# Reduciendo la complejidad y el esfuerzo

## Modularidad

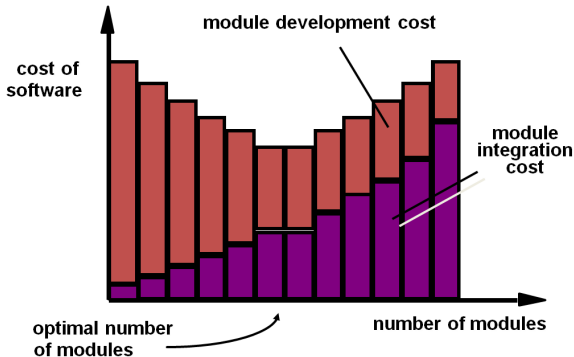
Permite reducir la complejidad y el esfuerzo de desarrollo  
(**divide y vencerás**):

- Sea  $x$  el problema a resolver
- Sea  $C(x)$  la función que mide la complejidad del problema  $x$
- Sea  $E(x)$  la función que mide el esfuerzo para resolver  $x$
- Si  $p_1$  y  $p_2$  son problemas y  $C(p_1) > C(p_2)$ , entonces:
  - $E(p_1) > E(p_2)$
  - $C(p_1 + p_2) > C(p_1) + C(p_2)$
  - $E(p_1 + p_2) > E(p_1) + E(p_2)$

# Modularidad vs costo de desarrollo

## Modularidad

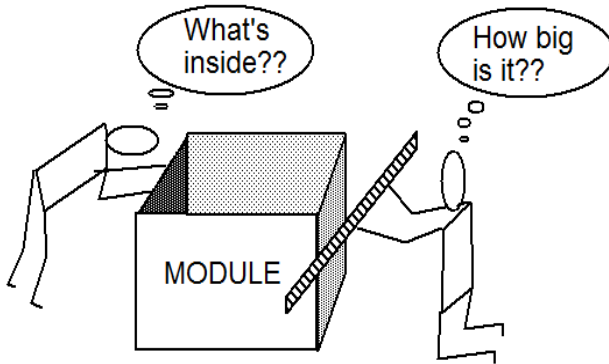
*¿Cuál es el número "correcto" de módulos para un diseño de software específico?*



# Tamaño de los módulos

## Modularidad

La siguiente imagen resume dos tipos de criterios:





# Independencia funcional

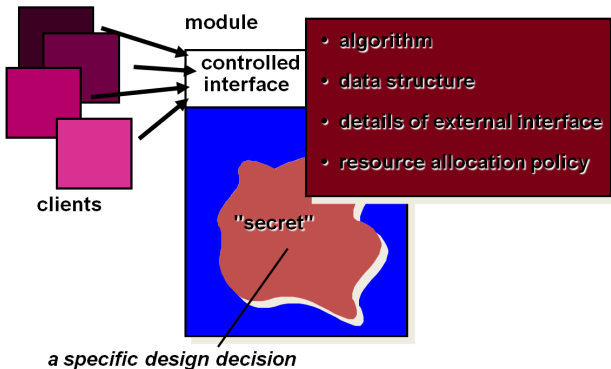
- El concepto de **independencia funcional** es una **derivación directa** del de **modularidad** y de los conceptos de **abstracción** y **ocultamiento de la información**
- El **desarrollo y mantenimiento** de software es **más fácil** utilizando **módulos independientes**
- **Mide** la **independencia entre módulos** utilizando **dos criterios**:
  - **Acoplamiento**: Medida de la interdependencia entre los módulos
  - **Cohesión**: Medida de la fuerza funcional relativa de un módulo

◀ Volver a conceptos de diseño

# Ocultamiento de información

- Cada módulo **oculta** los **detalles internos** de sus procesos **comunicándose** con otros módulos a través de **interfaces** bien definidas
- Este concepto suele denominarse ***encapsulamiento***
- **Evita** la **propagación de errores** que pudiera producirse al hacer cambios en algún módulo
- Permite **ocultar** detalles tales como la **estructura de datos** o los **detalles internos** de implementación
- Puede utilizarse como **técnica de diseño arquitectónico** o como **criterio de modularidad**

# Ocultamiento de información



## Refinamiento por pasos

- Propuesto por Niklaus Wirth, es un **enfoque de diseño descendente** en el que el **desarrollo** de una aplicación se realiza mediante **refinamiento sucesivo** de los niveles de **detalle procedimentales**
- **Actividades:**
  - Tomar **decisiones de diseño** para **descomponer en niveles elementales**
  - **Aislar aspectos** de diseño que **no** sean totalmente independientes
  - **Posponer decisiones** sobre detalles de representación
  - **Demostrar** que en cada iteración el **refinamiento refleja** fielmente los pasos **anteriores**

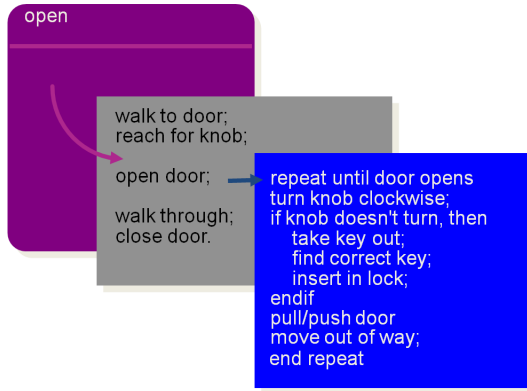
# Refinamiento por pasos



## Ventajas

- Descomposición descendente
- Añade los detalles poco a poco
- Aplaza las decisiones que pudieran resultar más controvertidas en el diseño
- Verificación continua de la consistencia del diseño

# Refinamiento por pasos



# Clases de diseño

- El **modelo de análisis** define un conjunto de **clases de análisis** que describen completamente los elementos del dominio del problema
- Durante el diseño se define un conjunto de **clases de diseño**:
  - **Refinando las clases de análisis**: se añaden detalles de diseño que permitirán la posterior implementación de las clases
  - **Produciendo nuevas clases de diseño** que permitan el desarrollo de la solución

# Clases de diseño



## Se sugieren cinco tipos diferentes:

- Clases de **interfaz** con el usuario
- Clases del **dominio del problema** (refinamiento clases de análisis)
- Clases del **proceso** (menor nivel de abstracción del problema)
- Clases **persistentes** (almacenamiento de datos)
- Clases del **sistema** (gestión y control del software)

◀ Volver a conceptos de diseño

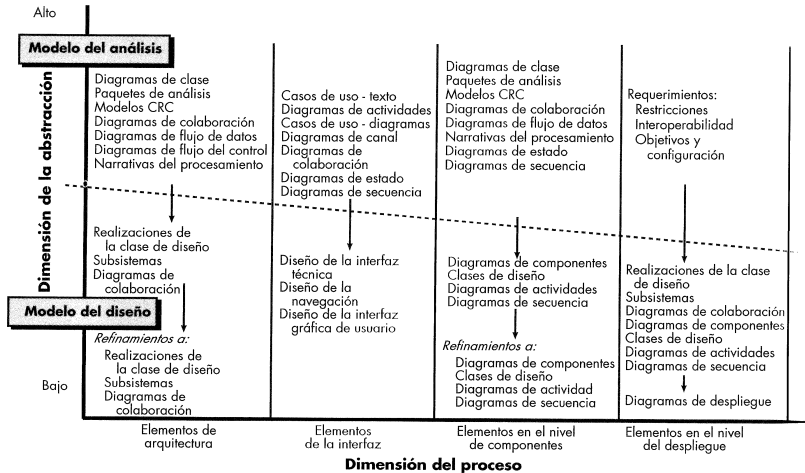


# Características

## Modelo del diseño

- El **diseño** es un proceso constituido por **fases** que sintetizan **representaciones** de la **estructura de datos**, la **estructura del programa**, las características de la **interfaz** y los detalles de los **componentes** a partir de los requisitos de la información
- Independientemente de la metodología utilizada, el diseño se desarrolla a partir del siguiente **conjunto de actividades**:
  - Diseño de datos/clases
  - Diseño arquitectónico
  - Diseño de la interfaz
  - Diseño de componentes

# Dimensiones del modelo de diseño



# Diseño de datos/clases

- El diseño de datos (o arquitectura de datos) **crea un modelo de datos** o información que se representa con un **alto grado de abstracción**
- **Después**, este modelo de datos **se refina progresivamente** en representaciones asociadas a una implementación específica y que pueden procesarse mediante el sistema informático
- En muchas aplicaciones de software la **arquitectura de los datos** tendrá una profunda **influencia** sobre la **arquitectura del software** que los procesa y sobre la **calidad del software**
- La **estructura de los datos** constituye una **parte importante** del **diseño de software**

# Diseño arquitectónico



## Definición (Bass, Clements y Kazman, 2003)

*La arquitectura del software de un programa es la estructura o estructuras del sistema, lo que comprende a los componentes del software, sus propiedades externas visibles y las relaciones entre ellos*

- Es el **equivalente al plano de una casa**, que proporciona una visión global de la casa. Los elementos del diseño arquitectónico dan una **visión general del software** y de sus funcionalidades

# Diseño arquitectónico

- La arquitectura **no es el software operativo**
- Es una **representación que permite**:
  - 1 **Analizar la efectividad del diseño** para cumplir los requisitos del sistema a desarrollar
  - 2 **Considerar alternativas arquitectónicas** en una etapa en la que las **modificaciones del diseño** aún son **asequibles** de abordar
  - 3 **Reducir los riesgos** asociados a la construcción del software

# Diseño arquitectónico

- El modelo arquitectónico se obtiene a partir de tres fuentes:
  - 1 La información del dominio de la aplicación del software que se va a elaborar
  - 2 Los elementos del modelo de análisis específico (diagramas de representación elaborados y clases de análisis), así como sus relaciones y colaboraciones
  - 3 La disponibilidad estilos arquitectónicos y sus patrones

◀ Volver

# Diseño de la interfaz

- Los elementos de **diseño de la interfaz** del software permiten que la **información fluya** hacia dentro y fuera del sistema, y definen cómo están **comunicados** los **componentes** que forman parte de la **arquitectura**
- En **UML** se define **interfaz** del siguiente modo: *"Es un especificador para operaciones visibles desde el exterior (públicas) de una clase, un componente u otro clasificador (incluso subsistemas), sin especificar su estructura interna"* (OMG 2003)
- En resumen, una **interfaz** es un **conjunto de operaciones** que **describen** alguna **parte del comportamiento** de una **clase** y dan **acceso a aquellas**

# Diseño de la interfaz

- Existen **tres elementos importantes** del diseño de la interfaz:
  - 1 La **interfaz de usuario**: incorpora **elementos estéticos** (distribución, color, gráficos, mecanismos de interacción ...), **elementos ergonómicos** (distribución y colocación de la información, navegación por la IU...) y **elementos técnicos** (patrones de la IU y patrones reutilizables)
  - 2 Las **interfaces externas** que actúan sobre otros sistemas, dispositivos, redes y otros productores o consumidores de información
  - 3 Las **interfaces internas** que involucran a los distintos **componentes** del diseño



# Diseño de componentes

- Es **equivalente** al conjunto de **especificaciones** para cada **habitación de una casa**: describe por completo el **detalle interno** de cada **componente** de **software**
- Permite **transformar** los **elementos estructurales** de la arquitectura del software en una **descripción procedimental** de los **componentes** de éste:
  - Define **estructuras de datos** para todos los objetos de datos locales
  - Expresa **detalles algorítmicos** para todo el procesamiento que ocurre dentro de un **componente**
  - **Detalla** una **interfaz** que permite el acceso a todas las operaciones de los **componentes** (comportamientos)

# Diseño de componentes

- Los **detalles del diseño de componentes** se pueden modelar a **distintos grados de abstracción**:
  - El **procesamiento lógico** se puede representar con **diagramas de actividad**
  - El **flujo detallado de funcionamiento de un componente** se puede representar mediante **pseudocódigo u otro tipo de diagramas**

◀ Volver

## Bibliografía



R. Pressman

*Ingeniería del Software. Un Enfoque Práctico. 7ª Ed.*  
McGraw-Hill, 2010



L. Debrauwer

*Patrones de diseño para C#. Los 23 modelos de diseño: descripción y soluciones ilustradas en UML 2 y C#*  
Ediciones ENI, 2012